

Microsoft GraphRAG with an RDF Knowledge Graph - Part 1

Source: <https://medium.com/@ianormy/microsoft-graphrag-with-an-rdf-knowledge-graph-part-1-00a354afdb09>

Published: 2024-08-12T17:52:23Z

PDF generated from reader view on 2026-06-27 15:55 UTC

Using a local LLM & Encoder to do Microsoft's GraphRAG

Image 1: Ian Ormesher (https://miro.medium.com/v2/resize:fill:32:32/2*SR5X4bF4sxSh9EjoygqzMw.jpeg)

5 min read

Aug 12, 2024

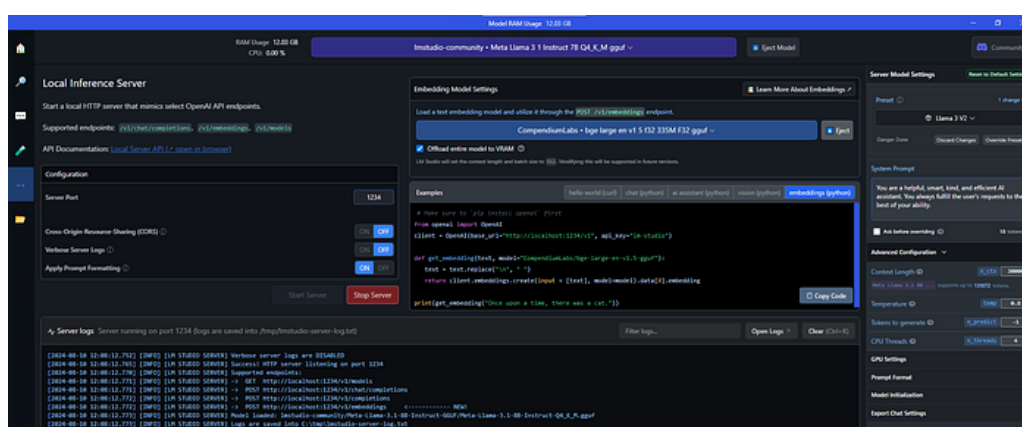


Image 2

LM Studio running locally

Like a lot of people I've read Microsoft's paper about GraphRAG and wanted to have a go at running this. However, it requires a lot of calls to an OpenAI compatible model which can quickly prove to be quite expensive. Having heard about the ability to run these models locally without incurring costs I thought I'd have a go at doing this myself. My steps are (which I'm also planning to blog about):

- Part 1 - Setup a local OpenAI compatible server with Chat and Embeddings endpoints and use that with Microsoft GraphRAG.
- Part 2
(<https://medium.com/@ianormy/microsoft-graphrag-with-an-rdf-knowledge-graph-part-2-d8d291a39ed1>)
- Take the output from Microsoft GraphRAG and upload it into a Knowledge Graph on an RDF Triple Store (I'll use GraphDB)
- Part 3
(<https://medium.com/@ianormy/microsoft-graphrag-with-an-rdf-knowledge-graph-part-3-328f85d7dab2>) -
Use that populated Knowledge Graph to produce better results for RAG queries (using SPARQL 1.1)

I know there are already quite a few blog posts about doing the first part, so if you're familiar with that and have produced your outputs, then please skip this post and jump to Part 2.

Local Server Requirements

In choosing a locally run server I had a number of requirements that fitted in with the way Microsoft's GraphRAG worked:

- Able to use GGUF formatted models
- Endpoints for both Chat and Embeddings
- Open source
- Able to run on my local machine
- No passing of my data to an external website

GGUF Format

Why did I specify the requirement for GGUF formatted models? The LLM models are true to their name - they're very large! When you save the trained model together with its weights the files produced are a lot of Gigabytes in size. To reduce this size they can be converted to GGUF format. This not only produces a reduction in size but it also speeds up the loading of the models. See the resources list at the end of the article for more details about this.

llama.cpp

It seems that most of the servers that can be run locally are built on top of an open source product called llama.cpp (<https://github.com/ggerganov/llama.cpp>) . If you go to their website they give a list of UIs that use their product. This list is really useful as you know that they will all be able to use GGUF formatted models.

Get Ian Ormesher's stories in your inbox

Join Medium for free to get updates from this writer.

Remember me for faster sign in

I started with using Ollama as that is quite popular at the moment. But with the latest version you have to import your GGUF into their system where it gets converted into their own proprietary format. If you download files it's the same story. I wasn't keen on not being able to see GGUF files so I chose another UI from the list: LM Studio (<https://lmstudio.ai/>)

LM Studio

Having downloaded the latest version of LM Studio from their website here (<https://lmstudio.ai/>) I then downloaded a model to use for the Chat (I used **Meta-Llama-3.1-8B-Instruct-Q4_K_M.gguf**) and a model for the Embedding (I used **bge-large-en-v1.5-f32.gguf**). Both of these produced good results as I'll show in my next blog post.

Configuration

There are some things that need to be changed in order for the models to work well with Microsoft GraphRAG. Once you've downloaded the models we'll use, choose to load them:

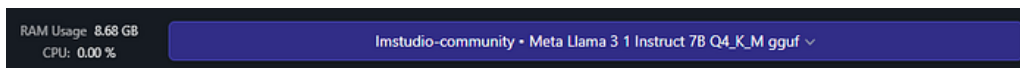


Image 3

Loading the model

On the right-hand side you will see the settings for the server. When it loads the model it sets these values and also chooses some default settings. It's one of the defaults we'll need to change - namely the context length. As models continue to improve and get bigger and bigger, they also allow a larger context length. When you load the model it will know the maximum size this can be. But the default is only 400 - which is quite small. I made mine 30,000 in order to cope with the large chunk sizes of text we could be dealing with (the default is 12,000 for the GraphRAG):

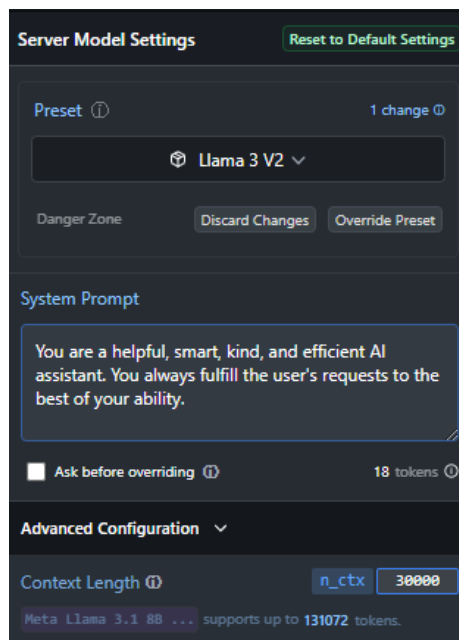


Image 4

Server Model Settings

Next step is to load up the embedding model:

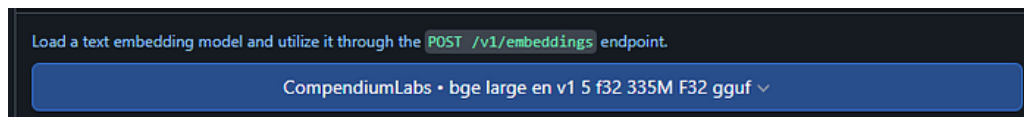


Image 5

Load the embedding model

Now the stage is set for us to setup Microsoft GraphRAG and run it using this local server.

Running Microsoft GraphRAG

If you follow the helpful "Getting Started" (https://microsoft.github.io/graphrag/posts/get_started/) page and have created your initial environment you then need to tailor 2 of the files produced in your "ragtest" folder.

.env File

This usually stores the API key for your OpenAI account. For our setup we simply need to change the key value to "lm-studio":

settings.yaml

The important things to change in this file are:

- That we are using the OpenAI chat API (lines 6 and 36)
- The name of the LLM model (line 7)
- The maximum number of tokens (line 9)
- The API base (LM Studio shows this - line 11)
- Number of threads. I set this to 1 because I was also running LM Studio on the same machine (line 27)
- Embedding model (line 37)
- Chunk Size. The new default is 1200. I set it to the old value of 300 which produces better results (line 51)

All that's left is for us to kick off the GraphRAG with the following command :

```
python -m graphrag.index - root ./ragtest
```

Watching the prompts

If you want to see more about the chat prompts that Microsoft GraphRAG is producing you can simply turn the "verbose" switch of LM Studio to on:

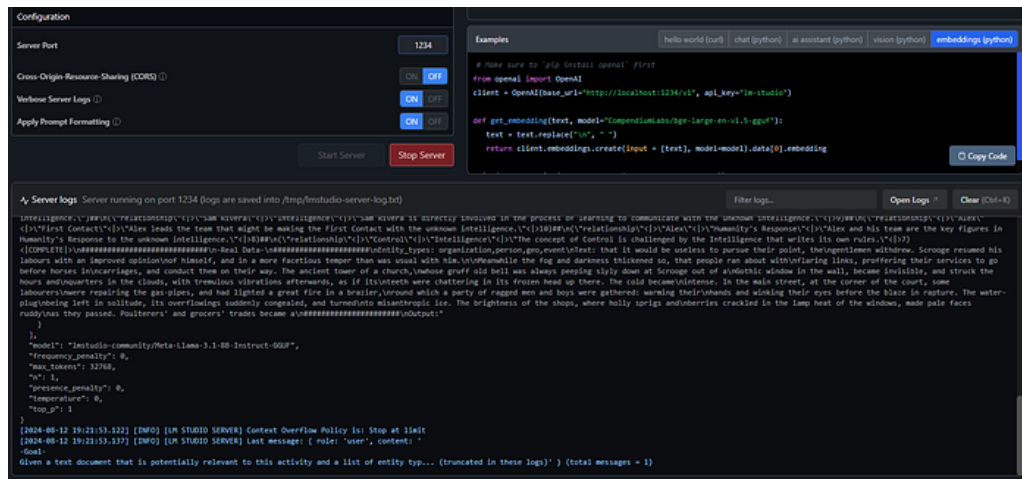


Image 6

verbose setting showing chat prompts

Once the GraphRAG has run and the corresponding output files have been produced, you're ready to proceed to Part 2 and create an RDF Knowledge Graph.